Eberhard Karls Universität Tübingen
Mathematisch-Naturwissenschaftliche Fakultät
Fachbereich Informatik
Arbeitsbereich Kommunikationsnetze

Bachelor Thesis Computer Science

Overview and Performance Evaluation of Selected Highspeed Packet Processing Technologies on Linux for x86 Architectures

Lukas Hackl

28.06.2024

Reviewers

Prof. Dr. habil. M. Menth
Fachbereich Informatik
Arbeitsbereich Kommunikationsnetze
Eberhard Karls Universität Tübingen

Supervisor

Fabian Ihle M.Sc.

Lukas Hackl:
Overview and Performance Evaluation of Selected Highspeed Packet Processing Technologies on Linux for x86 Architectures
Bachelor Thesis Computer Science
Eberhard Karls Universität Tübingen
Time period: 01.03.2024 - 28.06.2024

Erklärung

Hiermit versichere ich, dass ich die vorliegende Bachelor Thesis selbstständig und nur mit den angegebenen Hilfsmitteln angefertigt habe und dass alle Stellen, die dem Wortlaut oder dem Sinne nach anderen Werken entnommen sind, durch Angaben von Quellen als Entlehnung kenntlich gemacht worden sind. Diese Bachelor Thesis wurde in gleicher oder ähnlicher Form in keinem anderen Studiengang als Prüfungsleistung vorgelegt.

Tübingen, den 28.06.2024

(Lukas Hackl)

Contents

1 Introduction **2**

2 Technical Background **3**
2.1 Network Processing Paradigms . . . 3
2.1.1 The Kernel Space . . . 3
2.1.2 extended Berkeley Packet Filter . . . 4
2.1.3 The User Space . . . 4
2.1.4 The Python Library Scapy . . . 5
2.1.5 The Kernel Bypass . . . 5
2.1.6 Snabb . . . 6
2.2 Network Interface Technologies . . . 7
2.2.1 Raw Sockets . . . 8
2.2.2 NIC-Specific Drivers . . . 8
2.2.3 TUN/TAP Interfaces . . . 8

3 Related Work **9**
3.1 General Network Function Virtualization . . . 9
3.2 Network Function Virtualization Performance Comparisons . . . 10
3.3 Traffic Generation . . . 11

4 Implementation **12**
4.1 Network Functions . . . 12
4.1.1 Packet Return . . . 12
4.1.2 Routing . . . 13
4.1.3 Encapsulation . . . 15
4.2 Implemented Network Processing Paradigms . . . 16
4.2.1 User Space Implementation . . . 16
4.2.2 Kernel Bypass Implementation . . . 17
4.3 Optimizations . . . 19

5 Evaluation and Results **20**
5.1 Overview . . . 20
5.2 Environment . . . 20
5.3 Python Benchmarks . . . 23
5.4 Snabb Benchmarks . . . 25
5.5 Kernel Bypass with Multiple Flows . . . 30
5.6 User Space vs Kernel Bypass . . . 31

6 Discussion and Future Work **32**

7 Conclusion **34**

Bibliography **35**

List of Figures **38**

List of Tables **39**

This thesis investigates and evaluates selected high-speed packet processing technologies deployed on Linux for x86 architecture. The selected technologies, namely eBPF in the kernel space, Snabb for kernel bypass and a traditional implementation in user space using python, are introduced and their performance examined regarding packet processing tasks. Through three custom benchmarks (packet return, routing and encapsulation) the efficiency of each technology is evaluated through which their applicability and performance in different scenarios is shown. The kernel bypass implementation with Snabb showed an substantial increase in performance, in high-speed packet processing scenarios with multiple traffic flows and traffic rates of up to 100 Gbps, when compared to traditional implementations in user space in all cases. The thesis provides a comprehensive overview of the performance of high-speed packet processint technologies and highlights the potential of the kernel bypass technique.

1 Introduction

In our current rapidly evolving digital time there is an ever growing demand for high-speed packet processing. Because there is more and more data that is stored and processed the data traffic rates are also increasing at the same speed together with increasing complexity of network architectures. Fast, efficient and reliable packet processing is therefore necessary for maintaining optimal performance in a network. In these complex networks Linux is widely used as the operating system and therefore plays an important role in the packet processing on x86 architectures. An exciting technique for processing packets is Network Function Virtualization (NFV) [10]. It can be used as an alternative to traditional hardware solutions. Hardware solutions often require additional middle-boxes for specific use-cases and packet processing tasks which makes them expensive and inflexible as you require new hardware for each specific task. Through the advances in commodity hardware it is possible to reach almost equal performance on x86 hardware when compared to proprietary solutions regarding network operations [9]. Because of this, high-speed packet processing technologies can be implemented in software and used as a replacement for expensive middle-boxes at a fraction of the cost [12] while offering more specific configurations more flexibility and an easier to manage infrastructure [25]. In recent years many techniques and frameworks for implementation of such network functions have emerged which offer a high performance in packet processing [31]. This thesis investigates selected high-speed packet processing technologies on Linux for x86 Architectures. The selected techniques are a traditional implementation in user space, a implementation directly in kernel space and the usage of kernel bypass. These techniques use different approaches on how they process packets at different parts of the operating system. The underlying fundamentals of the network processing paradigms, together with the selected high-speed processing technologies will be introduced and explained and lay the basis for the implementation and evaluation part of the thesis. For all introduced techniques, three network functions are implemented that serve as benchmarks. With these benchmarks the performance of each technique is evaluated. In order to provide a comprehensive overview of the performance of each technique, traffic is generated with different bandwidths and frame sizes. The performance of each network function is evaluated in comparison to one another, as well as the performance of each high-speed packet processing technique. The results of the evaluation are then described and explained, followed by a discussion of the resulting overview. Lastly an outlook for future work is presented before the thesis concludes.

2 Technical Background

This chapter introduces and explains the fundamentals necessary for understanding the topic and the following chapters of the thesis.

2.1 Network Processing Paradigms

Through operating systems the connection between computer hardware and software applications is established, enabling communication and coordination. This includes the management of resources, processes, memory, file systems and devices. For a more secure and stable performance, modern operating systems divide their address space into two parts, namely the kernel space and the user space. Additionally in the last years a technique called kernel bypass emerged which keeps the advantages of the user space while bypassing the disadvantages of the kernel space. These network processing paradigms, together with a selected high-speed packet processing technique in each respected space are introduced and explained in the following sections.

2.1.1 The Kernel Space

The Kernel space is a privileged space in the system memory of an operating system where the kernel resides [16] [30]. The kernel is the core component of the operating system, which manages system resources, essential services and facilitates communication between hardware and software components. The kernel has direct and unrestricted access to the underlying hardware and performs essential tasks like memory allocation, device driver management and handles interrupts which is important regarding packet processing. Because the kernel has unlimited access to all resources, operations performed directly in kernel space have a low latency and can be executed fast and without communication or asking for permission of any other part of the operating system. Access from user space to the kernel is extremely controlled and only possible through specific interfaces through which services or resources are requested from the kernel. Everything that is executed inside the kernel must be working perfectly as even small errors in operations can lead to a complete system failure. Therefore kernel development is very demanding concerning tests and very time consuming since no errors are allowed here. Regarding packet processing another problem arises inside the kernel. The kernel contains the so called network stack which includes all processes and protocols when it comes to networking, like TCP/IP, UDP and the necessary device drivers. Each packet that arrives at the kernel has to traverse the complete network stack although not everything from the stack is necessary for a specific operation on a packet, resulting in an overhead.

2.1.2 extended Berkeley Packet Filter

A framework which avoids this problematic and works directly in the kernel space is extended Berkeley Packet Filter (eBPF) [8]. It was originally developed by the Linux community and has evolved into a powerful framework for high-speed packet processing. eBPF works with a virtual machine (VM) which executes code at byte-level directly in the kernel. The VM itself offers a safe environment for user-written programs without weakening the stability or security of the kernel. eBPF programs work by attaching them to so called hooks inside the kernel where events at different layers can then be intercepted and processed. Packets can therefore be intercepted inside the networking stack so they do not have to traverse it entirely but only the parts which are necessary for the intended operation. This removes the overhead and increases the speed at which the packets can be processed dramatically. eBPF also uses just-in-time (JIT) compilations to translate the byte code directly into native machine code which further reduces the overhead. Another feature of eBPF is that eBPF programs can be dynamically loaded and unloaded into the kernel, while traditional kernel modules do not offer this feature. This flexibility allows continuous experimentation and changes of the program without interrupting system operations. Besides packet filtering and packet manipulation, eBPF has expanded far beyond its original purpose and is also used for tasks like monitoring or performance analysis. The eBPF project has shown what is possible for direct kernel programmability and continues to grow.

2.1.3 The User Space

The user space is the part of the address space where programs and applications concerning users run [30]. It is completely isolated from the kernel space and has no direct access to the hardware. Every interaction with the underlying hardware and resources of the system, such as reading from or writing to files, allocating memory or performing network communication is managed over system calls to the kernel space. If a user program is executed on a computer it is normally executed in user space. Through this separation in the address space security is guaranteed and user-written programs can be tested and experimented with, without risking a complete system failure. If it fails in user space there are no negative effects on the kernel or hardware. Therefore, the development of user space programs is much safer and takes less time as the underlying operations of the programs are managed and executed by the kernel. Programs in user space often use libraries which are precompiled code for a specific functionality and save time in development. Network applications can also run in parallel to each other as each application operates within its own memory space. The big disadvantage of programs that run classically in the user space and communicate with the kernel is that they are very slow regarding packet processing. The constant communication with the kernel for the purpose of accessing hardware and resources, as well as the network stack inside the kernel results in an increased overhead and slow down the processing of packets immensely. This ultimately leads to a considerable decline in performance. For purpose of comparison, in this thesis a classical approach to packet processing directly in the user space is implemented in Python to show the significant differences in speed between the selected techniques.

2.1.4 The Python Library Scapy

Scapy [4] is a powerful library for packet processing and packet manipulation. Custom packets can be crafted by defining headers, payloads and many other parameters for specific requirements or use-case. Packets can also be sniffed, dissected, extracted or directly manipulated on their way to their destination. It is often used for network penetration tests or network scanning and is an extremely versatile tool as it offers a great functionality and is easy to use. In the case of this thesis it was used to do simple manipulation of a packet, for example sniffing a packet and interchanging its source and destination IP address.

2.1.5 The Kernel Bypass

As the name suggests, kernel bypass is a technique to bypass the operating systems kernel when processing or handling packets [27]. Traditionally, packets are passed from the user space to the kernel and go through multiple layers inside the kernel, like the networking stack, different protocols and device drivers. This creates a big overhead and limits performance as not every layer is specifically needed to process packets in a specific case. By performing kernel bypass, it is possible to gain direct access to the underlying hardware from user space, therefore using all the advantages of user space and eliminating the disadvantages without direct hardware access and therefore the constant dependency on the kernel for hardware access. This can increase performance in high-speed networking applications manifold. To understand how this is accomplished a look into how packets are normally handled is necessary. In the traditional case where the kernel is involved, incoming packets are announced at the NIC by so called interrupts. If an interrupt is received by the system, the CPU interrupts its current task and jumps to a specific interrupt handler routing for the received interrupt. In networking this would normally be the interrupt handler of the device driver. The incoming packet is then read and processed through the networking stack. Once finished, the CPU returns to its original work. This is beneficial in non-high-speed packet processing cases, as an incoming packet is announced by the interrupt and its not necessary to constantly check for new packets which would waste CPU resources. In a high-speed packet processing scenario this becomes a big disadvantage. As the interrupts are irregular its hard to plan. There are a lot of context-switches as the CPU constantly switches between tasks and interrupt handling therefore creates a big overhead when large amounts of packets are incoming. This is solved in kernel bypass by accessing the hardware directly from user space with so called polling mode drivers (PMDs) [6]. PMDs allow direct access to the NIC registers and memory through which packets can be directly worked with. Instead of waiting for interrupts from the NIC, the PMD constantly polls the NIC to check for new packets. This is done in a loop in defined intervals. Hereby it is important that system resources are managed efficiently as the CPU is constantly involved while polling. To minimize CPU usage during idle periods has a high priority when programming PMDs. As the polls are occurring

in constant intervals, packets can be processed more efficiently. This is because the packet processing can be scheduled more effectively, with fewer context switches occurring. Furthermore, the overhead of the kernel and its networking stack can be completely neglected.

2.1.6 Snabb

A framework that performs kernel bypass, like described in Section 2.1.5, is Snabb [14]. Snabb is an open source, high-performance networking framework. It was started by L. Gorrie and is developed since 2012. It was designed for building high-performance networking applications by using network functions virtualization and is written in Lua which is a very efficient scripting language, easy to learn and known for its simplicity and efficiency. With the Lua language, there is also LuaJIT used which is a Just-In-Time (JIT) compiler for Lua. Through this, the performance is boosted as the Lua code is compiled into native machine code at runtime. The JIT compilation translates the byte code of lua into highly optimized machine code, which increases the performance compared to regular Lua compilation. Additionally Snabb is capable of using Foreign Function Interface (FFI). This allows code written in one programming language to call functions and use data type defined in another language. In Snabb this allows Lua to interact with C libraries to perform low-level memory operations. This makes it possible to implement high performance code that can interact directly with the hardware and make system calls. Snabb is built with a modular approach in mind. Networking applications or network functions are built using so called *Apps* [13] which perform specific networking tasks such as packet processing or traffic management for example. An App can perform the same tasks as a hardware device. Input and output links are configured manually and can be as many as necessary but all are unidirectional. For one input and one output two links have to be set on an app.

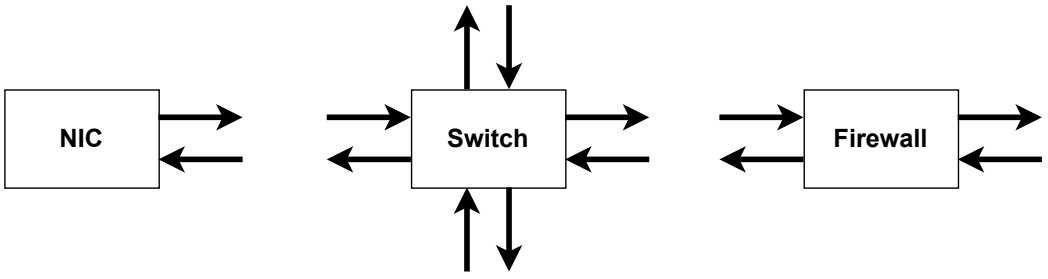


Figure 2.1: Exemplary Snabb apps [13].

These apps are reusable building blocks which combined, implement a complete network function. The modular apps are linked together to create an so called *App Graph* [13] which represents the network functions and performs more advanced tasks with its underlying apps.

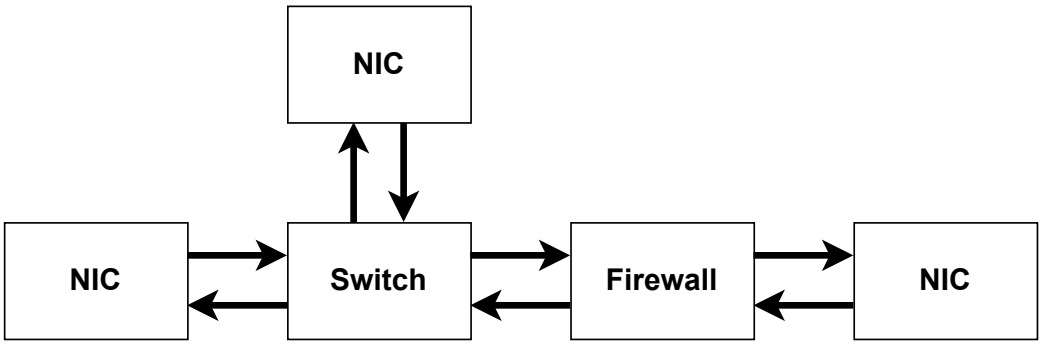


Figure 2.2: Exemplary App Graph [13].

Through this modular approach huge App Graphs can be built which realize complex functionalities in the network and all in software without costly hardware devices. An App Graph is executed through the Snabb engine which processes each app in so called "breadths" [11]. Each app has a pull and a push function, which are executed alternating by the engine. The pull function of an app introduces a specific amount of packets into the App Graph. Normally, only one pull function is executed as only one app of an app graph is responsible for introducing packets. After the pull function, the push function of every App in the App Graph is executed consecutively in the order that they were configured and linked inside the App Graph. The push function realizes the actual functionality of the network function. Each push function of an app performs the packet processing or manipulation tasks to achieve the desired result. This can be applying filtering rules on packets, changing the payload or headers of the packet or routing between networks to only name a few simple cases. The engine alternates between the pull and push function in a loop as long as the network function is being executed by the engine. As a selected amount of packets is introduced with each pull. The processing of packets is therefore plannable and minimizes the overhead created for each packet. Performance-wise, Snabb is capable of using multi-core parallelism and efficient memory management, therefore making it possible to create a high throughput of data and low latency's in packet processing operations.

2.2 Network Interface Technologies

Network Interface Cards (NICs) provide an interface for sending and receiving data over the connected network [7]. They work on the physical and data link layer of the OSI model and manage transmission and formatting of data, so that the data can be processed in the computer or sent over a network. Modern NICs offer more features like load balancing or network traffic prioritization. An operating system is using software components, so called drivers, to communicate with the hardware NICs. The drivers translate commands from applications into low-level instructions which can be used and executed by the underlying hardware [22]. Generic drivers, which work for many NICs, are sufficient for general networking tasks but offer

low performance high-speed processing tasks. Therefore NIC-specific drivers are implemented. These drivers are optimized for a particular NIC and can improve the performance significantly. Other methods for network communication are raw sockets and TUN/TAP interfaces. These are explained in the following sections.

2.2.1 Raw Sockets

Packets that arrive at the NIC are passed through the data link layer and are then handed off to the network layer. Raw sockets make it possible to capture packets at this transition before they pass through the entire TCP/IP stack, and therefore bypassing higher-level protocols [28]. Through this headers and data can be manipulated with minimal interference from the network stack.

2.2.2 NIC-Specific Drivers

NIC-specific drivers are drivers that were programmed specifically for a NIC hardware model. They are optimized for the underlying hardware of the NIC, therefore offering stable and reliable operations through the NIC which is not always the case with generic drivers. Through these custom drivers maximum performance and efficiency can be achieved as advanced features like hardware offloading where specific functions are executed directly by the hardware instead of following the standard process through software functions. These drivers are hard to implement but absolutely necessary for maximum performance. NIC-specific drivers are also implemented in frameworks to offer optimized performance for a specific NIC model. Snabb for example has drivers for different hardware NICs from Intel, Mellanox and Broadcom which are optimized for the usage of Snabb programs.

2.2.3 TUN/TAP Interfaces

TUN (Network TUNnel) and TAP (Terminal Access Point) are virtual network interfaces [18]. A TUN simulates a network layer device and communicates through IP packets. A TAP driver simulates a data link layer device and communicates through Ethernet frames. The interfaces are connected through a bridge to a NIC to capture incoming packets. Specifically a TAP interface is ideal for the use with NFV like virtual switches or virtual routers as they perform operations at the data link layer and need to handle a wider range of traffic types than applications that only interact with IP traffic at the network layer.

3 Related Work

This chapter reviews significant contributions from recent research relevant to this thesis.

Regarding the field of high-speed packet processing on Linux, several studies have been conducted, exploring and evaluating various technologies and frameworks. These studies help to understand the performance and applicability of different approaches in network data plane programming and packet processing with network function virtualization.

3.1 General Network Function Virtualization

Han et al. [15] provide an in-depth exploration of Network Function Virtualization. It lays a fundamental understanding for the usage of NFV and describes them as an transformative approach for network service provisioning, as the decoupling of software implementation from hardware platforms can reduce costs and energy consumption, as well as offering more tailored services. Several use cases such as virtualization of mobile core networks or home networks are discussed and challenges are identified associated with the adoption of NFV. The authors call for further research to overcome these challenges to ensure a successful integration of NFV into existing network structures.
Mijumbi et al. [23] give a detailed survey on the concept of Network Function Virtualization. They discuss the significance of NFV in transforming telecommunication service provisioning by decoupling network functions from physical devices, as well as its benefits, current state and future research challenges. Additionally they review key NFV projects, standardization efforts, early implementations and use cases.
Abdelwahab et al. [1] explore the potential of Network Function Virtualization to improve the viability of 5G radio access networks. The paper describes how NFV can increase the efficiency and flexibility through decoupling service design from service implementation which allows 5G operators to activate feature per service rather than in the whole network. They also demonstrate how, through the decoupling of software from hardware, costs can be reduced. Different use cases are discussed and various open research problems are highlighted related to the implementation of NFV in 5G.
Mirjalily and Luo [24] provide a comprehensive survey of Network Function Virtualization and Service Function Chaining. The paper highlights the potential, of both technologies, to transform network service deployment by offering more flexibility and lower costs. It is discussed how service chains are defined and how functions are linked together. Furthermore they describe where the functions are placed in

the network and how they are scheduled to achieve optimal performance. They emphasize the need for advanced algorithms to fully realize the benefits of NFV and SFC.

3.2 Network Function Virtualization Performance Comparisons

Scholz et al. [27] give an in-depth and thorough overview over Snabb. They focus on the overall architecture including user-space packet processing, the usage of LuaJIT for performance optimizations and techniques like the kernel bypass. Real-world applications, specifically the applicability for telecom networks with projects like Deutsche Telekom's Terastream [29], are also highlighted. A performance evaluation of Snabb is done with sample applications provided in Snabb directly. The performance is compared with similar tests of other processing frameworks. No custom apps or functionalities are implemented for evaluating the performance like in it is done this thesis.
Zhang et al. [31] evaluate the performance of seven high-speed software switches, including Snabb, for Network Function Virtualization. For the performance evaluation, they create a comprehensive benchmark methodology which is independent of the control plane used by the software switch to enable a fair comparison between the techniques (OVS-DPDK, t4p4s, FastClick, Snabb, Berkeley Extensible Software Switch, Vector Packet Processing, VALE). They provide a fundamental understanding of the architecture, self-contained or modular, and specific features for each software switch. In the performance comparison they use four scenarios (point-to-point, point-to-virtual, virtual-to-virtual and loopback), measuring throughput and latency in each scenario. The software switches forward packets and are linked together as service function chains but there are no custom implementations of specific packet manipulation tasks like adding a header or manipulating packet data.
Lettieri et al. [20] give another comprehensive analysis of various I/O frameworks for high-speed packet processing with Network Function Virtualization. They mention more of the challenges encountered in achieving high performance network speed in virtualized environments due to the limitations of network stacks and traditional I/O. Several frameworks (SR-IOV, OVS-DPDK, NetVM, Netmap, Snabb) are examined, comparing features, flexibility and overall performance. The selected frameworks are also evaluated against criteria important for NFV, such as dependency on specific NIC models, portability of VM images, CPU utilization and more. The performance is measured by forwarding packets through different network topologies. The changes in the experiment occur in the overall setup of the network and not directly in different operations on the packets themselves.
Ara et al. [2] introduce a benchmarking framework for comparison of the performance of different kernel bypass mechanisms in regards to achieving high-performance in inter-container communications. Several virtual switches (DPDK Basic Forwarding Sample Application, OVS, VPP, Snabb, SR-IOV), implemented with kernel bypass technologies, are evaluated and their performance is compared to each other. The test scenarios include several configurations with different container pairs, packet

sizes and sending rates. The performance is measured in regard to throughput, latency and scalability for packet processing between different containers on the same and also on different physical machines. Custom sender/receiver and client/server applications are used to generate network traffic and are deployed with containers, but no direct manipulation of packets is taking place.
Barbette et al. [3] investigate high-speed packet processing frameworks in user space. They use the Click modular router [17] to review and compare various frameworks demonstrating the increased performance with kernel bypass techniques over kernel based packet processing solutions. The focus in this work is on leveraging hardware features like multi-queues, multi-core processors and non-uniform memory access to increase performance of software packet processing solutions.

3.3 Traffic Generation

Lindner et al. [21] present P4TG, a high-performance P4-based traffic generator. It runs on the Intel Tofino™ASIC and is capable of generating and analyzing network traffic at speed up 1Tb/s, distributed across 10 x 100 Gbps ports. P4TG offers traffic generation with customizable parameters such as traffic rate and frame size and supports both constant bit-rate as well as random traffic generation. Sending and receiving rates are measured directly in the data plane recording data like packet loss, packet reordering and round-trip times. Additionally P4TG can be used to analyze the network measuring input traffic and forwarding while recording similar data. Because it achieves nanosecond accuracy it is perfect for use in high-speed packet processing scenarios and is used in this thesis to generate traffic for the implemented benchmarks.

4 Implementation

In this section the implementation of the network functions is shown. Also the reason for choosing each specific benchmark is explained.

4.1 Network Functions

In order to evaluate the performance of the chosen high-speed packet processing techniques, it is necessary to demonstrate the significant performance increase that kernel-bypass offers. The difference in performance between simple and more complex operations on packets with different techniques has to be shown, as the performance of these operations should increase significantly between the selected techniques. To provide a comprehensive assessment of several critical aspects of packet handling, three benchmarks were chosen. The implementation of these, which serve as the core of the later evaluation, are described and shown in detail in the following sections.

4.1.1 Packet Return

The packet return benchmark performs the least complex operation of the three benchmarks. It involves sending a packet to a NIC, receiving the packet, extracting the source and destination IP addresses from the IP header, and then switching these addresses so that the packet's new destination is the original source, and the original destination becomes the new source. The manipulated packet is then sent back to its source. This fundamental operation serves as a baseline for measuring the performance of packet processing by assessing raw throughput. It helps to understand how efficiently the chosen packet processing technique handles minimal packet manipulation and the associated overhead in such a straightforward operation.

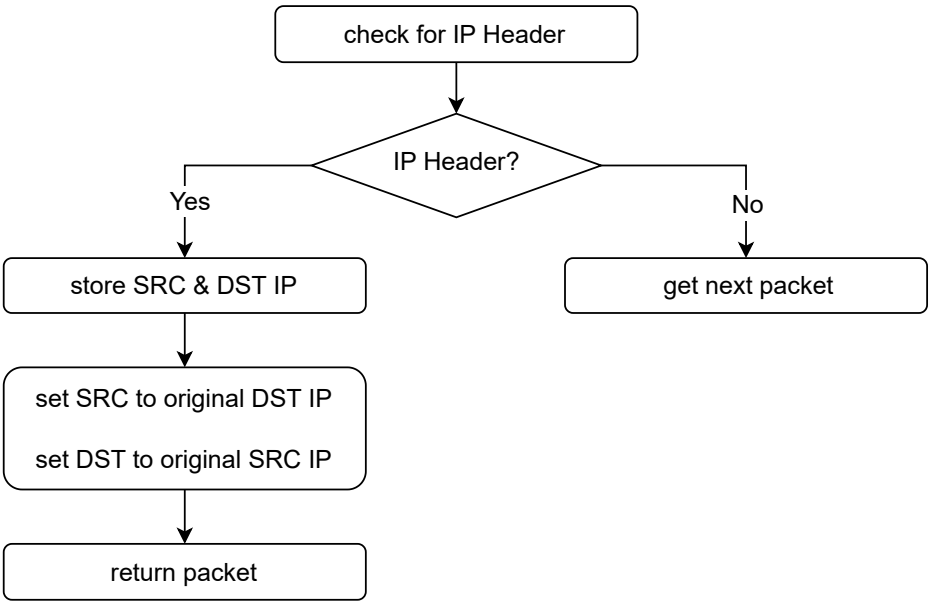


Figure 4.1: Flowchart of packet return network function.

4.1.2 Routing

In the routing benchmark, the packet itself is not directly manipulated. Instead, the primary operations involve searching for the correct route within the routing table.

When a packet is received, its destination address is extracted. Using this destination address, the network function iterates through the routing table, which contains three entries in each row: network address, subnet mask, and egress port. Entries of the with the same network address but different subnet masks are in order of the length of the subnet mask. The entry with the longest subnet mask comes first and so on, so that longest prefix matching occurs automatically while iterating through the table. During each iteration, a logical AND-operation is performed between the extracted destination address and the subnet mask of the current row, filtering out the network address of the destination IP. If the result matches the network address in the current row, the corresponding egress port is identified as the correct route. The packet is then sent through this egress port without any manipulation or change. This benchmark focuses on the operations surrounding the packet, involving more

Network Address	**Subnetmask**	**Egressport**
10.0.0.0	255.255.0.0	eth-1
192.168.178.0	255.255.255.0	eth-2
192.168.0.0	255.255.0.0	eth-3
...	...	...

Table 4.1: Exemplary routing table.

complex data structures and algorithms for route lookup, rather than direct packet manipulation. As routing is a critical function in networking, the efficiency of these route lookup operations is essential. The performance measured in this benchmark directly relates to the effectiveness of routing in real-world applications, highlighting the stress on data structures and algorithms rather than the packet itself. The implementation of the routing benchmark is inefficient because it was kept extremely simple. Normally so called Ternary Content Addressable Memories (TCAMs) [19] are used for performing lookups in tables. These are designed for extremely fast searches as they allow data to be accessed based on its content by taking a data pattern as input and returning the address where the specific pattern is stored. This makes them perfect for usage in high-speed packet processing scenarios. For the evaluation of the benchmark in this thesis the simple and inefficient implementation is not a problem as it is equally inefficient for all techniques.

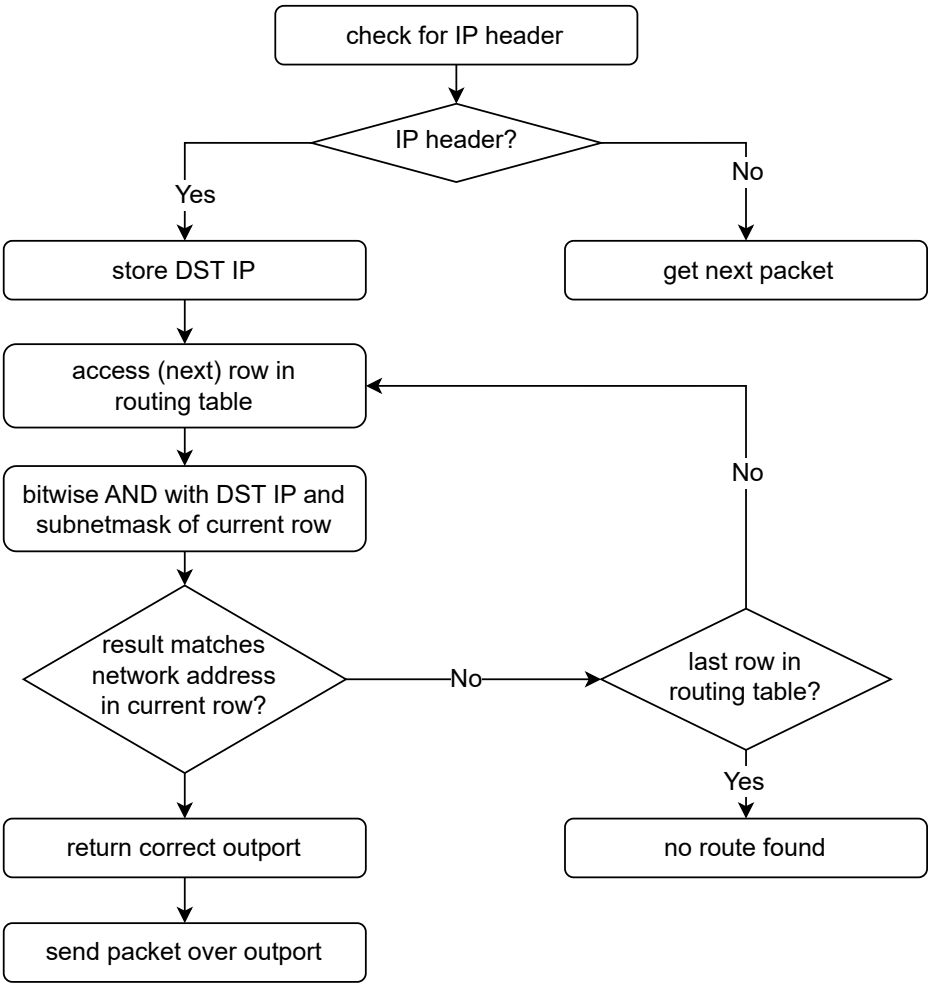


Figure 4.2: Flowchart of routing network function.

4.1.3 Encapsulation

The encapsulation benchmark involves directly manipulating a packet in a more complex way than the packet return benchmark. When a packet is received, it is disassembled into its headers and payload to create a new encapsulated packet. A new MPLS header is constructed and inserted between the Ethernet and IP headers of the original packet. The checksum is then updated because four bytes from the MPLS header were added. Once these operations are completed, the encapsulated packet is sent back. This benchmark demonstrates more complex packet manipulation by adding an MPLS header, which increases the number of steps compared to the simple packet return. Modifying headers and adjusting the packet length showcase the flexibility and overhead involved in the chosen packet processing technique. This additional complexity highlights the capability of the technique to handle more intricate operations, emphasizing the processing overhead introduced by the encapsulation process.

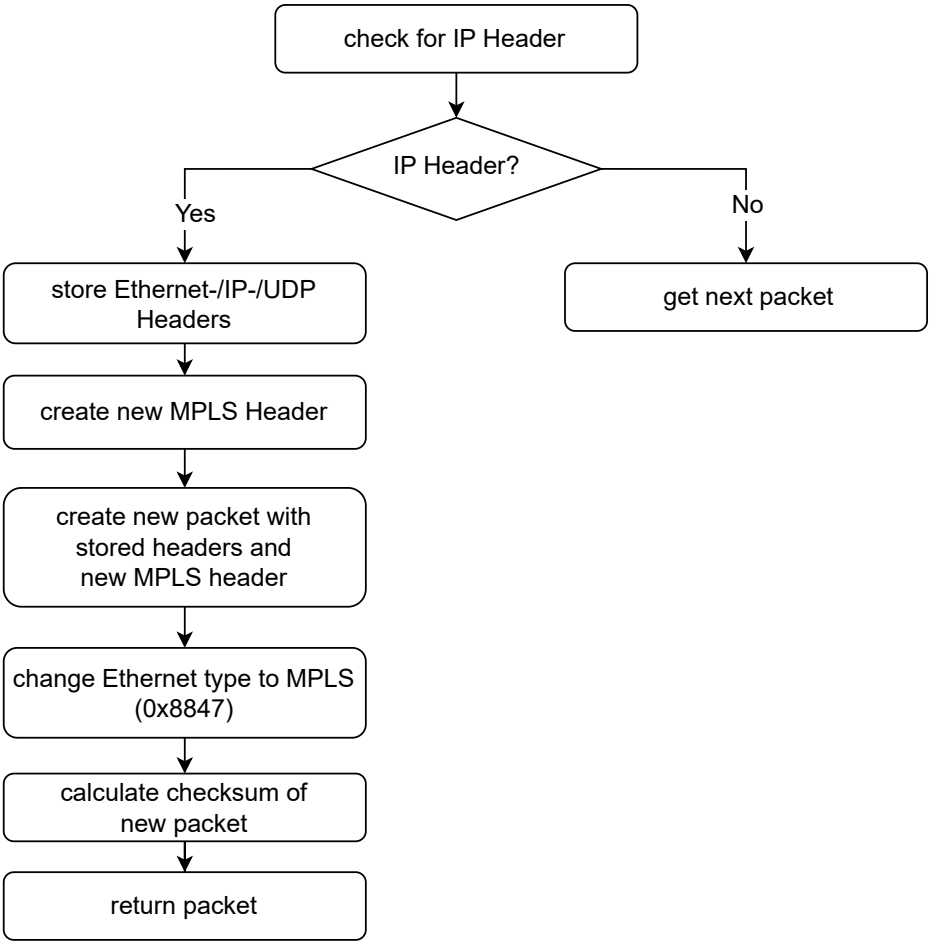


Figure 4.3: Flowchart of encapsulation network function.

4.2 Implemented Network Processing Paradigms

The following sections describe the implementation of the benchmarks from 4.1 in the selected network processing paradigms.

4.2.1 User Space Implementation

The implementation of the network functions in the user space was done in Python with the Python Library Scapy.
For the packet return benchmark a packet is inspected if it contains an IP header. If this is the case source and destination IP from the IP header are stored in variables. Then the stored source IP is assigned to the destination IP of the packet and the stored destination IP is assigned to the source IP of the packet, therefore switching the two. The packet is then returned. No problems were encountered in the implementation of the packet return benchmark.
For the routing benchmark a class is created which contains the routing table (Table 4.1) and the functions necessary to perform the switch functionality and determine the right port for an incoming packet. The routing table was implemented as a simple Python dictionary. The addresses inside the routing table are given in string format and are converted to an ipaddress format using the *ipaddress* library in Python. At first the conversion was done in a function. To optimize performance this was then changed to convert it directly inside the routing table. Therefore the conversion is only performed one time and not in every iteration through the table. Alongside the routing table the class contained three functions that realize the functionality. A `get_network` function, where the AND operation is performed to determine a network address. A `find_port` function which iterates through the routing table, using the `get_network` function to check for a given network address. And lastly a `route_packet` function which uses the `find_port` function to determine and return the right port for an incoming packet. Only minor difficulties regarding the correct conversion from string to ipaddress format occurred, otherwise the implementation was kept simple although therefore lacking performance like described in Section 4.1.2.
The encapsulation benchmark involves the insertion of an MPLS header into an incoming packet. Initially an attempt was made to insert an MPLS header directly into a packet and only changing the Ether type to MPLS. This was not successfull and returned a malformed packet every time. Consequently a different approach was adopted by storing the headers and payload of the packet, creating a new MPLS header and then creating a completely new packet with the stored information and newly added MPLS header. This method is successfull in achieving the desired result.
To connect the functions to the network three different approaches were taken to later compare their performance. The simplest method used is using the sniff function from Scapy, which captures incoming packets, and performing the network function on each captured packet. The second approach is using a raw socket with the *socket* library from Python. It was configured to capture packets directly from the network interface, capturing packets including the link-layer header and all incoming Ether-

net frames. Lastly a virtual TAP interface was implemented. Several approaches using different libraries were tested for the integration. Initially, attempts were made using the integrated TAP interface from the Scapy library, but other libraries performed better in this regard. The implementation in this thesis uses the *pytun* library. The virtual TAP interface is connected with a selected network interface through a bridge.

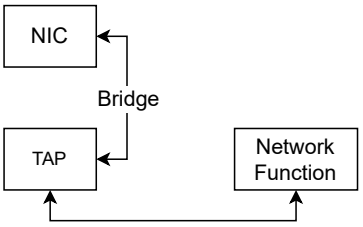


Figure 4.4: TAP interface setup.

The bridge is created at the start of the program, ensuring packets are forwarded from the network interface to the TAP interface and back when returning the packet. It is important to note that with the usage of the TAP interface one additional operation has to be implemented in the encapsulation process. In the Ethernet header the source MAC address has to be changed from the original MAC address to the MAC address of the configured TAP interface in order to correctly return the packet.

4.2.2 Kernel Bypass Implementation

The implementation of the benchmarks in Snabb is more complex. To directly access and manipulate packets the Ethernet and IPv4 headers have to be defined as C structs, containing all the fields and their sizes, using the FFI. This allows for manipulation of the packet data directly in memory without conversion, which is essential for high performance packet processing tasks. The related header lengths, necessary ether types and further data needed for processing such as composite types for an Ethernet header followed by an IPv4 header, or in the case of the encapsulation a composite type with an added MPLS header in between, are stored in constants. Furthermore the FFI is used to define utility functions such as conversions from IPv4 addresses to integers for operations on the IP addresses. To access and manipulate a packet, the packet data is cast into the related composite header type.
For the packet return benchmark the packet is cast into the ethernet header followed by the IP header. Through this the source and destination address could easily be accessed and switched to then return the packet. Once the above prerequiremental headers and utility functions are defined the implementation of the packet return benchmark is as straightforward as the one in the user space.
Regarding the routing benchmark, the routing table is implemented in a simple array filled with an array for each row of the routing table which contains the three entries like in Table 4.1. To work with the IP addresses, these are converted to integers

inside the routing table. The only conversion that is done inside the routing function itself is for the destination address of the incoming packet to later perform the bitwise AND operation with the subnet mask from the routing table. For the conversion from IP addresses to integers a simple function was tried at first which casts an address into a uint32 using the FFI. This approach was not robust enough, threw errors and converted IP addresses in strings to incorrect integers, so when converted back a different IP would show. A new function was therefore implemented which parses the IP address manually by performing bitwise operations to combine the four octets of the IP address into a single uint32. With this function the conversions and therefore the bitwise AND operation is successful. The bitwise operations were done with the Snabb library *bit*.
For the encapsulation benchmark the same approach as in the other two is used by defining the MPLS header as a struct. An MPLS header has fields of sizes 20, 8, 3 and 1 bits. The host byte order of x86 processors is litte endian so in order to send the new packet over the network this has to be converted to network byte order (big endian). To convert the header correctly from little to big endian the htonl (host to network long) function can be used, but would have to be written specifically for the MPLS header, as the htonl function only works for 32 bit. To bypass this it was decided to simply define the MPLS header as one 32 bit field and set the bits manually for each field. With the defined header another composite type that includes the MPLS header between the ethernet and IP header is added. Using the same approach as for the other two benchmarks it was tried to cast an incoming packet into the defined structure to access the headers and manipulate the packet by inserting an MPLS header in between. It was tried to insert the MPLS header directly and also by storing the headers of the incoming packets and creating a new packet, parsing it into the correct structure and assembling the headers in the correct order but both methods failed and returned malformed packets. No solution was found for the problems so a different approach was used. In the final implementation bitwise shift operations are used to move the bytes to create space for the MPLS header, copy the Ethernet header to the beginning and inserting the values of the MPLS header in the created space directly in hexadecimal notation converted to network byte order.
Two different network accesses are implemented, a raw socket and a NIC-specific driver, in this case a Snabb Mellanox driver. Both accesses are implemented in Snabb as Apps which can be connected to the network functions that were created. The raw socket output link is connected to the input of the network function. The output link of the network function is connected to the input link of the raw socket to measure the manipulated packets that are coming back from the network function. For the Snabb Mellanox driver the implementation was slightly more complex. The configuration of the driver needs two parameters: the PCI address of the hardware NIC and an array of queues over which the packets are processed. Each queue has to be configured and linked with the network function. In the implementation there is only one queue for the driver and therefore only one queue has to be configured and connected to a network function which was successful. Additionally worker threads can be added through Snabb. Through this multiple CPU cores can be used to increase the performance manifold. For this, additional parameters like

the number of workers, cores and queues per worker are added. For each worker a function is called which configures the network function to use, creates and assigns the queues per worker and links each queue with the network function. This process is implemented in its own function.

4.3 Optimizations

It is possible to make further improvements to the implemented benchmarks. Particulary the routing benchmark implementation is not optimal and can be improved by using TCAMs for routing table lookup operations. A different approach was taken for the encapsulation benchmark in Snabb and it would be interesting to implement the other two benchmarks with the same approach and comparing performance between the different implementations. Snabb suggests various hardware tuning to increase performance [26]. The server was configured following these suggestions. In the BIOS energy efficiency settings are set to maximum performance as to not limit CPU performance in the evaluation and hyperthreading is disabled to reduce latency. Regarding the system settings the CPU is isolated and reserved for data plane usage in order to exclude interference from the Linux kernel.

5 Evaluation and Results

This chapter provides a comprehensive evaluation and describes the results of the implemented benchmarks to determine the performance of the selected high-speed packet processing technologies.

5.1 Overview

We focus on the evaluation of the performance through measuring the throughput. In Kernel-Bypass the throughput should increase substantially by avoiding the overhead added by the kernel in normal user-space packet processing. Through varying packet rates for these benchmarks, also the scalability of each chosen packet processing technique can be shown. In standard user space there should be limitations due to the added kernel overhead and context switches, whereas kernel-bypass techniques are specially designed to handle high packet rates very effectively.
The chosen benchmarks should therefore be able to serve for the evaluation to create an extensive overview over the performance in the chosen techniques as they cover a good range of simple and more complex packet processing tasks. It is expected that the packet return benchmark and the encapsulation benchmark should show the best performance, while the routing benchmark should perform worse. Regarding the Network Interface Technologies used, the TAP interface is expected to offer the best performance in user space. With the usage of kernel bypass the specific driver for the Mellanox NIC should perform much better than the raw socket. Through the evaluation it can be shown which method or technique is better for a variety of networking tasks regarding overall performance and scalability.

5.2 Environment

The evaluation of the selected high-speed packet processing techniques is done with the P4TG traffic generator [21]. A UDP traffic stream is generated to the server while the benchmark network function is running in the network and the returning receiving rate is measured and saved to determine the overall performance. Firstly only one flow is tested and then multiple flows. The server is equipped with a Xeon Gold 6134 CPU @ 3.20 GHz CPU, 128 GB RAM and a Mellanox ConnectX-5 to which the traffic generator is connected.

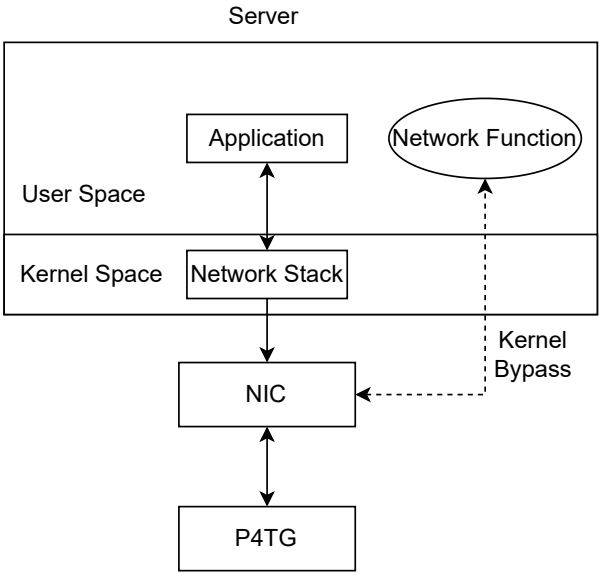


Figure 5.1: Environment for generating datasets.

Each benchmark is performed with four different traffic rates:

- 0.1 Gbps, 1 Gbps, 10 Gbps, 100 Gbps

For each traffic rate five different frame sizes are used:

- 64 bytes, 128 bytes, 256 bytes, 512 bytes, 1024 bytes

A single iteration consists of starting the traffic generator with a chosen traffic rate and a chosen frame size and measuring the receiving rate each second. This is performed until there are 11 datapoints. The first datapoint does not represent the actual performance as the traffic generator is only starting there and therefore the first datapoint is not representative. The first datapoint was cut from each set. This is repeated five times in total, resulting in 50 datapoints for a frame size with a specific traffic rate. For each of the five iterations the average receiving rate from the 10 datapoints is calculated, resulting in five average receiving rates for each of the 5 iterations for a single frame size. Then the average rate of the five average receiving rates is calculated. The resulting average of the average is saved as the final performance data for a frame size with a specific traffic rate. This process is done for all traffic rates with all frame sizes for each benchmark. Figure 5.2 depicts this process visually.

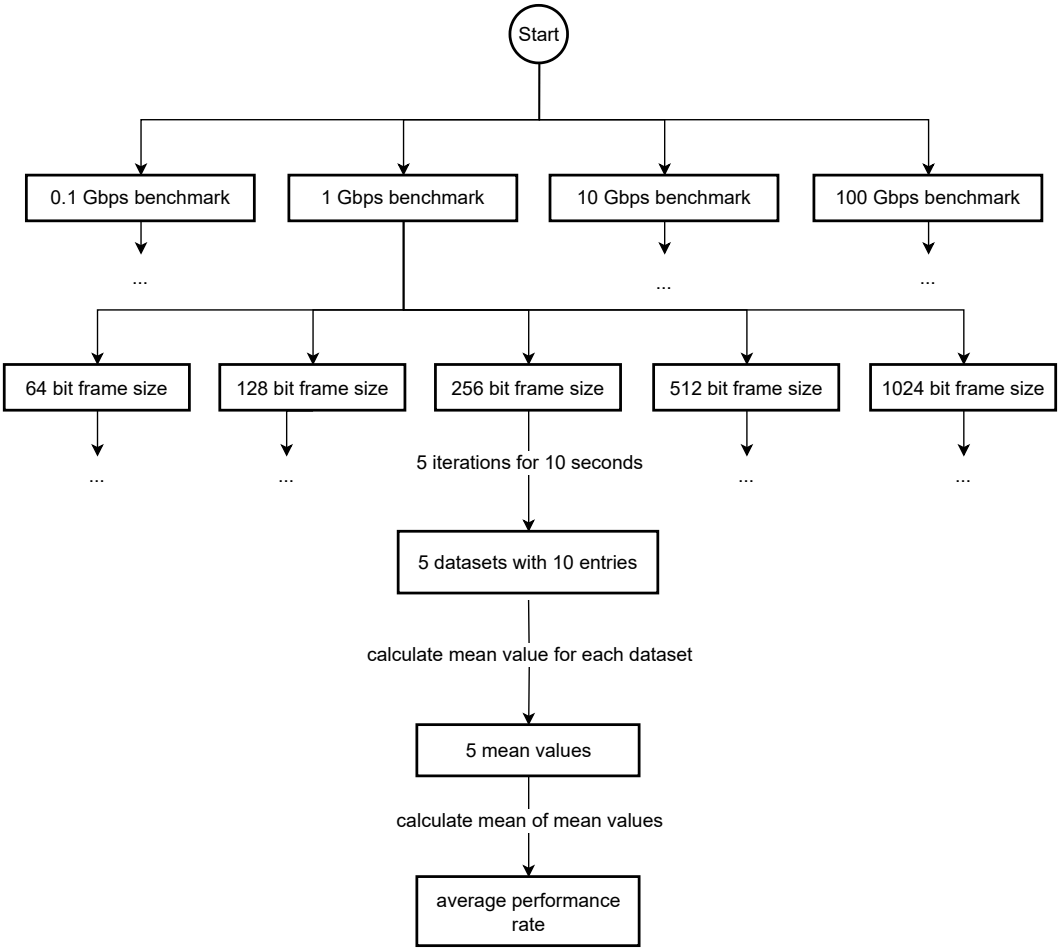


Figure 5.2: Process of generating datasets for each benchmark.

5.3 Python Benchmarks

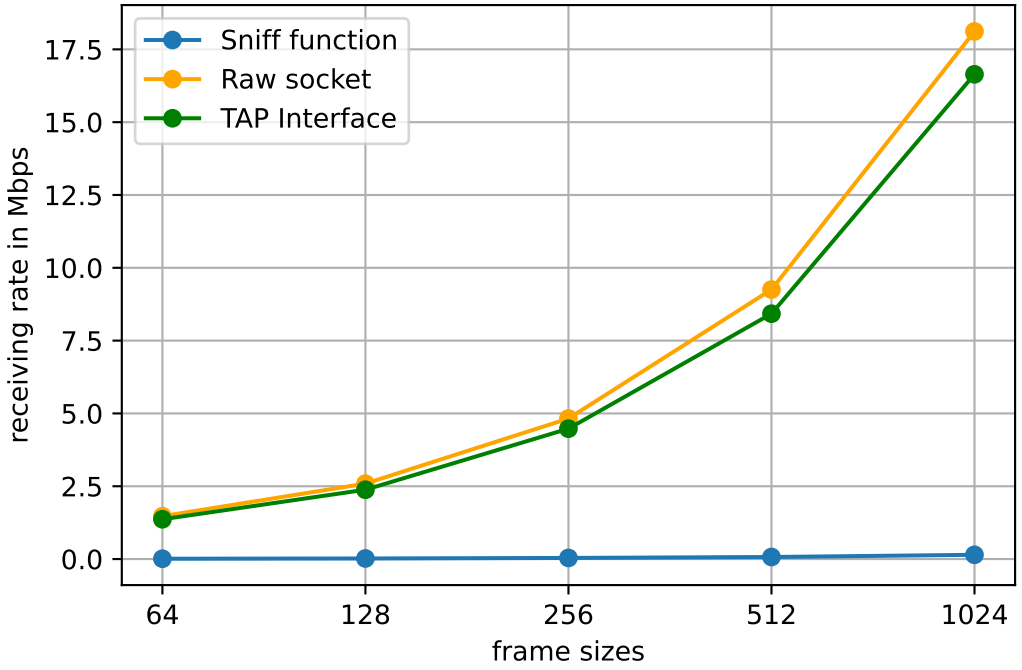


Figure 5.3: Performance of packet return benchmark in Python with different network interface technologies.

The data was initially generated using three different network interface technologies. Figure 5.3 shows the performance of the packet return benchmark when utilising a raw socket, a TAP interface and a basic sniff function with a transmission rate of 100 Mbps. It is noteworthy that even at relatively low transmission rates no network interface reaches a maximum reception rate with what should be the fastest benchmark. The results for the raw socket and TAP interface are similar, yet the sniff function is showing suboptimal performance due to its lack of optimization and traversal of the entire network stack. Because of the poor performance of the sniff function even in this simple case, only the raw socket and the TAP interface were evaluated further.

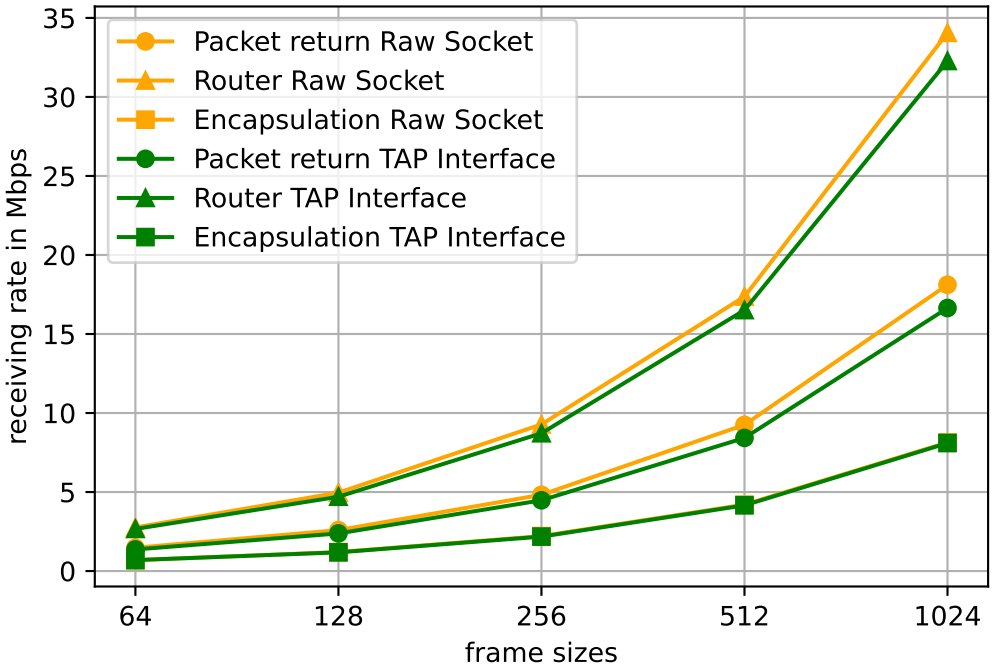


Figure 5.4: Python benchmark performance with Raw sockets and TAP interfaces.

It is visible in Figure 5.4 that the performance of the benchmarks is comparable between the raw socket and the TAP interface. However the raw socket performs slightly better in every case which was not expected. It can be observed that the throughput of the routing and packet return benchmarks increase approximately twofold when the frame size is doubled. As expected, the performance for lower frame sizes is worse than for bigger frame sizes. This can be attributed to two factors. Firstly, with lower frame sizes the CPU has to handle more packets per second. Secondly, each frame has a fixed overhead consisting of headers and footers. Consequently the mentioned overhead represents a greater portion of the total data being sent and processed when the frame size is smaller. Larger frames make more efficient use of the available bandwidth as there is less overhead per data. Surprisingly, the router benchmark has the best performance in Python with a maximum receiving rate of almost 35 Mbps with the usage of the raw socket. The TAP interface only lacks behind slightly. The packet return benchmark has a maximum receiving rate of around 17 Mbps. The encapsulation benchmark has the worst performance of the three with a maximum receiving rate of roughly only 0.7 Mbps. It seems like that in userspace direct packet manipulation, as it is done in the packet return and encapsulation benchmark, is more demanding than the lookup of a routing table. Still, the overall performance of the benchmarks, regardless of the network interface technology used, is unsatisfactory. Even with a relatively low sending rate, less than half of it is returned. This shows the limitations of conventional packet processing in user space, which is ultimately dependent on the kernel. All communication is

conducted via the kernel resulting in significant overheads. There is only a small room for optimizations directly from user space as for almost everything that would increase performance access has to be requested from the kernel. This limits the performance immensely.

5.4 Snabb Benchmarks

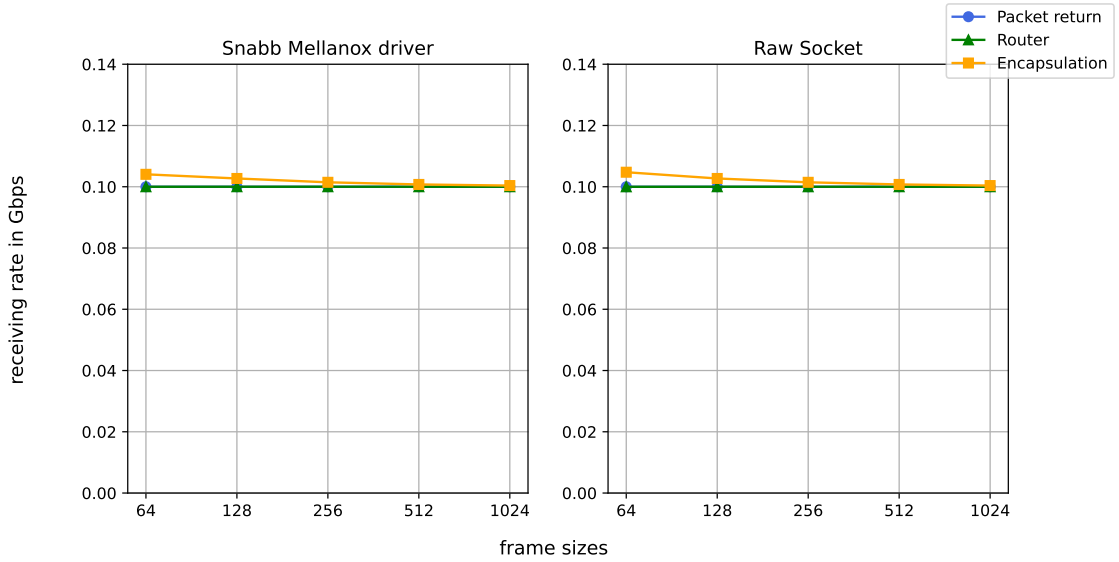


Figure 5.5: Snabb benchmark performance with one Flow at 100 Mbps sending rate.

Figure 5.5 shows that with a sending rate of 100 Mbps, both the Snabb Mellanox driver aswell as the usage of the Raw Socket achieve identical throughput. Both achieve the full receiving rate of 100 Mbps for all frame sizes. It is notable that the encapsulation benchmark has a higher receiving rate than sending rate, particularly for lower frame sizes. The addition of an MPLS header to each packet results in the packet being sent back with more bytes than before. At lower frame sizes more packets are handled per second, as explained in Section 5.3, which leads to the observed increase in the receiving rate. However, this effect declines with higher frame sizes, as fewer packets are handled per second and therefore fewer total bytes of MPLS headers are added.

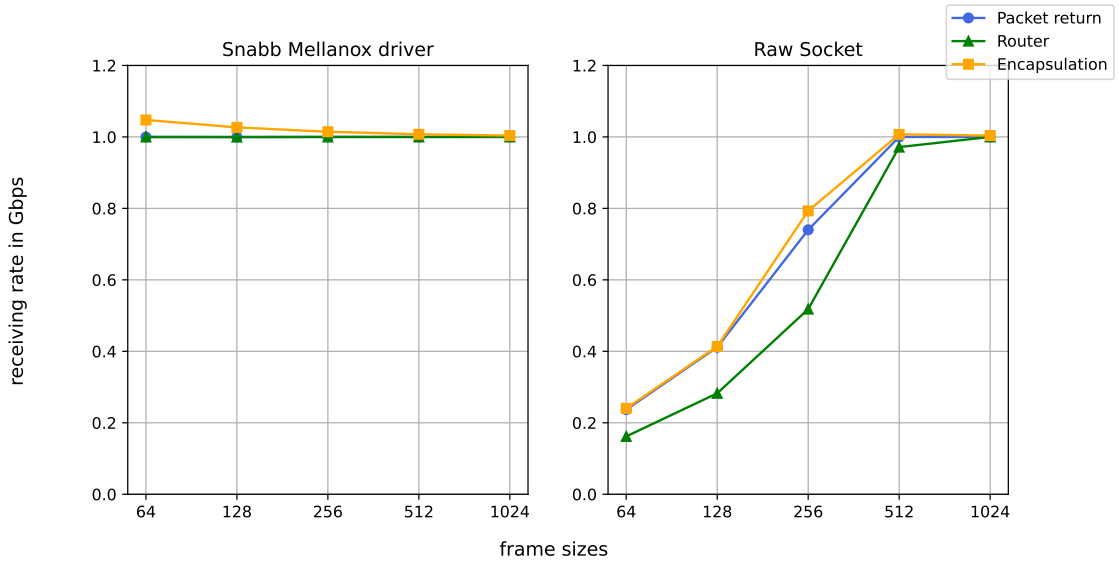


Figure 5.6: Snabb benchmark performance with one Flow at 1 Gbps sending rate.

In Figure 5.6 the sending rate is increased to 1 Gbps. The Snabb Mellanox driver continues to achieve the full receiving rate of 1 Gbps for all benchmarks with all frame sizes. The increased receiving rate for the encapsulation benchmark can still be observed. This is not the case for the Raw socket. The maximum receiving rate is only reached for the last two frame sizes of 512 bytes and 1024 bytes, for the packet return and encapsulation benchmark. In the router benchmark, the maximum receiving rate is only achieved with the largest frame size of 1024 bytes. It can again be observed that for the raw socket, the throughput is approximately doubled when the frame size is doubled, as explained in Section 5.3. At this sending rate, the lack of optimization of the raw socket becomes evident. The raw socket is not optimized for the used NIC and therefore cannot ensure efficient use of resources and suboptimal throughput. This results in a packet processing overhead that limits performance in comparison to the optimized driver for the NIC.

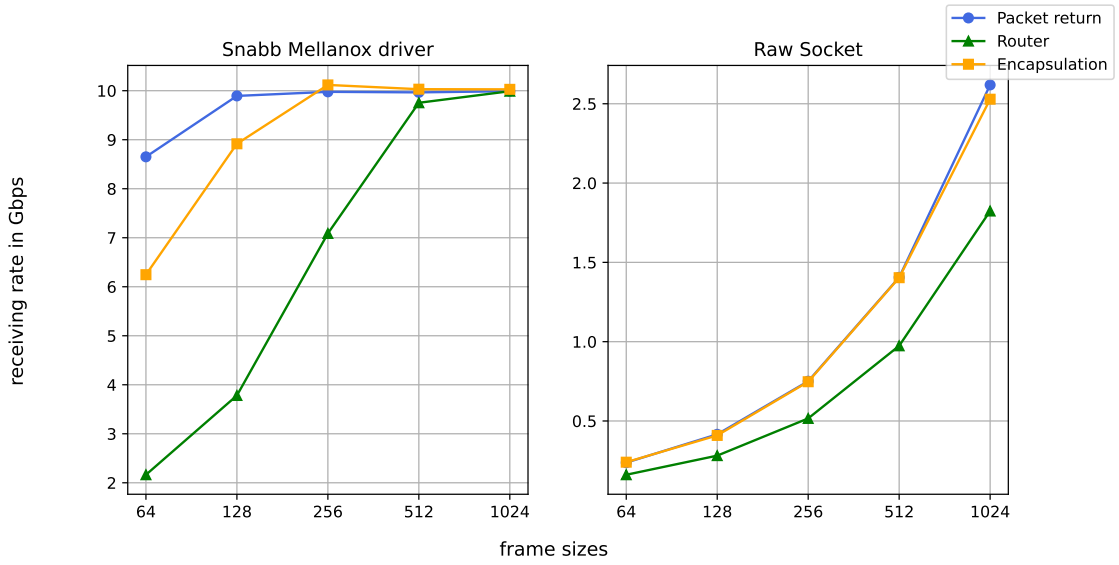


Figure 5.7: Snabb benchmark performances with one flow at 10 Gbps sending rate.

The performance differences become clearer with a sending rate of 10 Gbps, as shown in Figure 5.7. At this rate the Snabb Mellanox driver is unable to achieve the maximum receiving rate with all frame sizes. In the packet return and encapsulation benchmarks, the maximum receiving rate is achieved at a frame size of 256 bytes. Only at the maximum frame size of 1024 bytes that the router benchmark reaches 10 Gbps throughput. As previously observed, the performance can be seen to double approximately with each doubling of the frame size. With regard to the performance of the raw socket, its limit is reached at this point. The packet return benchmark and the encapsulation benchmark demonstrate the best performance and reach a maximum receiving rate of roughly 2.5 Gbps. The routing benchmark performs worse, reaching a maximum rate of roughly 1.8 Gbps. None of the benchmarks approach the potential maximum rate of 10 Gbps when utilising the raw socket. Although Snabb is working in polling mode by default and does not rely on interrupts, the raw socket is generally less optimized for packet processing. Due to the lack of hardware offloading, the CPU becomes a bottleneck and unable to keep up with the packet rates, resulting in the seen lack of performance.

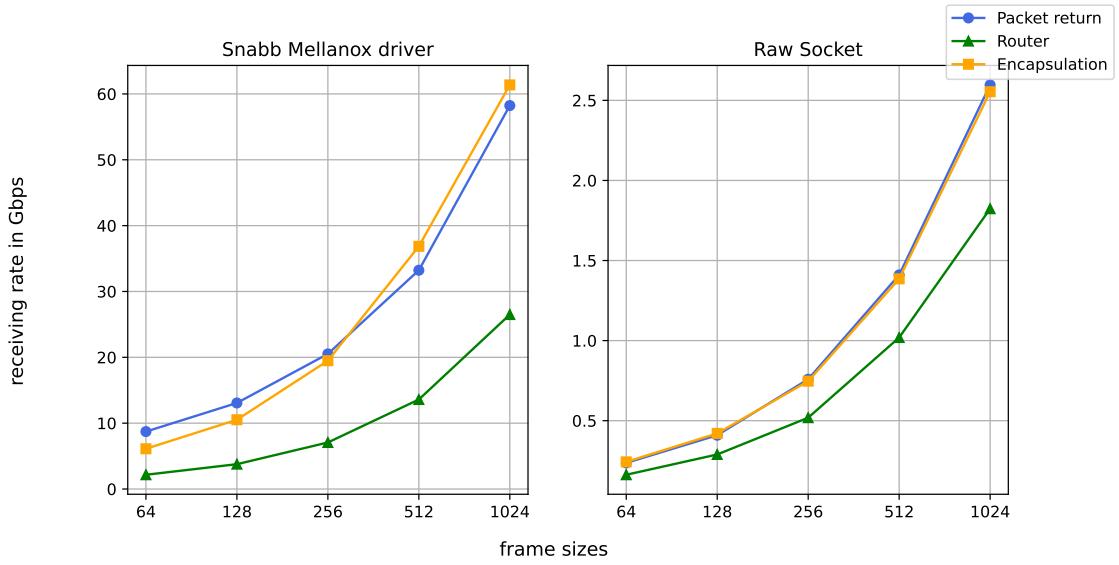


Figure 5.8: Snabb benchmark performances with one flow at 100 Gbps sending rate.

At an exceedingly high sending rate as depicted in Figure 5.8, it is not possible to achieve a full receiving rate with one single flow, even with the optimized driver for the mellanox NIC. At this point, the packet rate is so high that despite all optimizations and offloading mechanisms the packet rates are so high that the single CPU core tasked with handling the single flow is required to perform an exceedingly high amount of tasks, therefore limiting the performance and making it impossible to reach a full receiving rate. The packet return benchmark and the encapsulation benchmark achieve a receiving rate of approximately 60 Gbps, while the router benchmark reaches a maximum receiving rate of roughly 27 Gbps. The receiving rates of the raw socket remain unchanged in comparison to the performance with 10 Gbps, as this represents the maximum achievable rate. The graphs of both are quite similar and the pattern of doubling throughput if the frame size doubles can now also be seen with the Snabb Mellanox driver. It is notable that the Snabb Mellanox driver shows better performance in the encapsulation benchmark than the raw socket. The receiving rate is higher than the packet return benchmark in comparison to the raw socket as here the packet return benchmark is the fastest, which was expected. At a high sending rates with only one flow there are so many packets processed that even with the highest frame sizes the additional bytes added from the MPLS header in each packet are again noticeable and therefore show in the graph. As expected, the router benchmarks shows the poorest performance among the three under all sending rates due to the suboptimal implementation for lookup operations in the routing table as mentioned in Section 4.1.2.

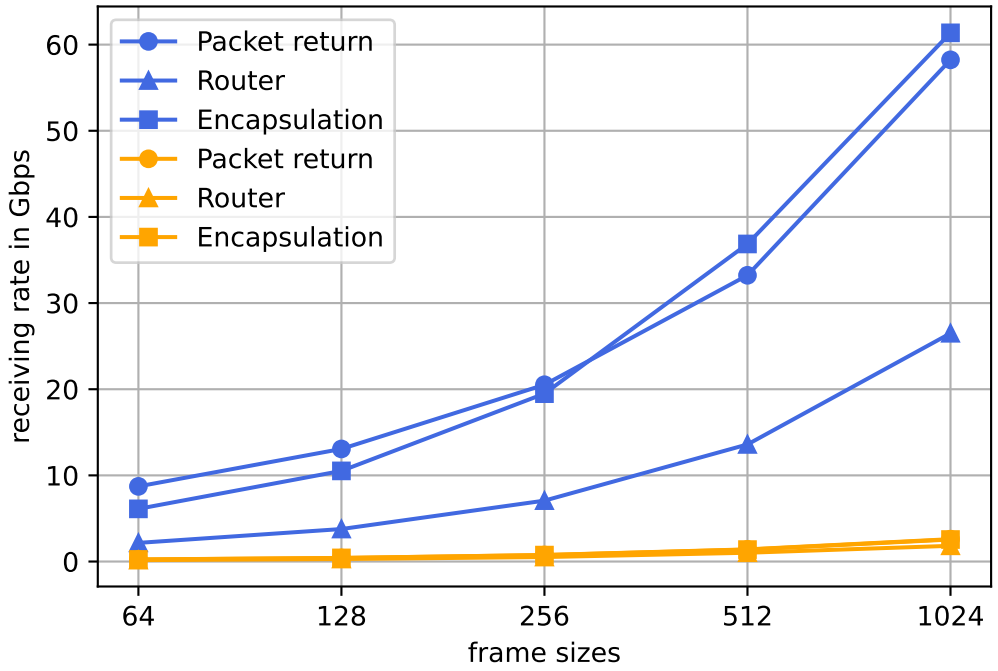


Figure 5.9: Snabb Snabb Mellanox driver performance compared to raw socket.

As the graphs in Figure 5.8 look very similar, differing only in the y-axis a better visualization of performance between the two network interface technologies was drawn in Figure 5.9. The figure demonstrates the superior performance of the Snabb Mellanox driver in comparison to the raw socket, which lacks most of the features and optimizations. This results in a significant increase in CPU load, leading to the observed poor performance.

5.5 Kernel Bypass with Multiple Flows

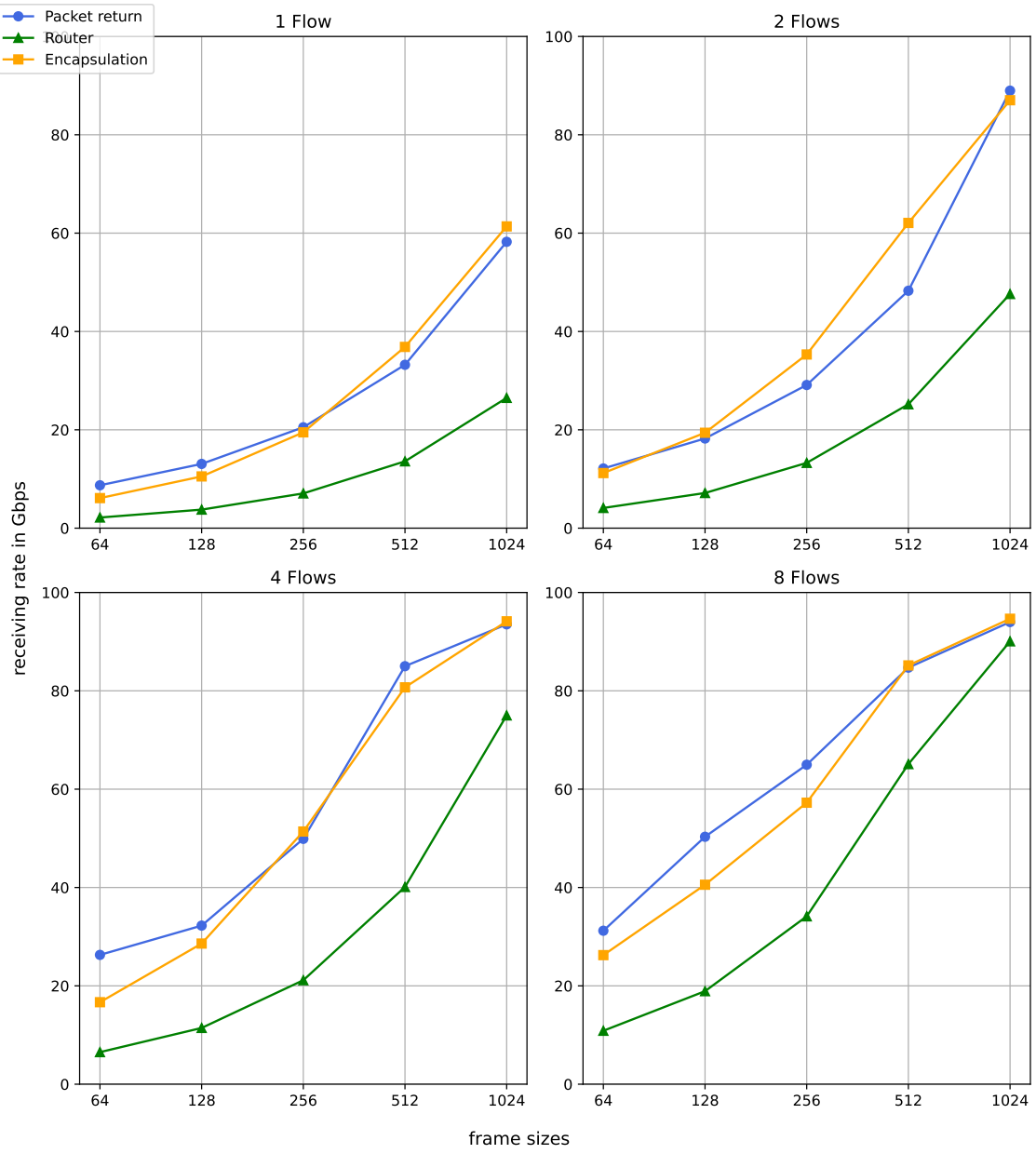


Figure 5.10: Snabb Mellanox driver performance with active worker threads.

Figure 5.10 illustrates the performance of the benchmarks with worker threads, that utilize CPU cores, and different numbers of flows. By increasing the number of workers the cores used by the CPU are also increased. For all iterations when generating the dataset there were 8 active worker threads with one queue each. Only the number of flows are changed in each iteration when generating the performance data. The highest receive rate achieved was a maximum of 94.65 Gbps for the encapsulation benchmark with 8 flows. Although it should be possible to reach the full 100

Gbps receive rate it was not detected what caused the packet loss. The performance with 1 flow is already known from Section 5.4. With 2 flows there is a noticeable increase in performance. Especially for larger frame sizes the performance increases drastically with both the packet return benchmark and the encapsulation benchmark almost reaching the maximum achievable receive rate. The routing benchmark lags behind due to its suboptimal implementation but still achieves a significant increase in performance. This becomes even more aparrent with four flows. Both the packet return and encapsulation benchmark reach their maximum with 93.99 Gbps and 94.65 Gpbs respectively. The routing benchmark reaches almost 80 Gbps with 4 flows. Performance at lower frame sizes also continues to rise steadily with more flows, showing a similiar pattern to the throughput/framesize relationship as the performance also roughly doubles when the number of flows is doubled. It is also noticeable that at 4 flows the performance graphs lose their curve slightly and become more linear as the large frame sizes reach their maximum possible performance. This is even more obvious at 8 flows where all cores of the CPU are active. The graphs are very linear. The routing benchmark almost catches up to the other to and achieves its maximum performance with 90.06 Gbps. The performance doubles again with larger frame sizes and the packet return and encapsulation benchmark reach their maximum performance at 512 bytes. As the number of flows increase, the performance also increases significantly. This demonstrates the scalability that Snabb and the kernel bypass technique are capable of in scenarios with fast increasing data loads.

5.6 User Space vs Kernel Bypass

	Packet return	**Routing**	**Encapsulation**
Python raw socket	0.018 Gbps	0.034 Gbps	0.007 Gbps
Snabb raw socket	2.59 Gbps	1.82 Gbps	2.55 Gpbs
Snabb Mellanox driver - 1 Flow	58.23 Gbps	26.49 Gbps	61.36 Gbps
Snabb Mellanox driver - 8 Flows	93.99 Gbps	90.06 Gbps	94.65 Gbps

Table 5.1: Best returning rate for each benchmark in User Space and Kernel Bypass at 1024 byte frame size and 100 Gbps traffic rate.

Table 5.1 clearly shows that using the kernel bypass is significantly faster than the user space implementation in Python. Even when using the raw socket the increase in performance is significant. When using the specific Snabb Mellanox driver the performance increase becomes dramatic, even more so when fully utilizing the CPU by implementing active worker threads and creating flows. As expected in Section 5.1, the kernel bypass technique increases the performance manifold through bypassing the entire kernel and using a optimized driver for the NIC. The substantial efficiency gains are clearly demonstrated making the technique ideal for high-speed packet processing tasks.

6 Discussion and Future Work

The results of this thesis highlight the significant performance differences between the selected high-speed packet processing techniques deployed on Linux x86 architectures. The evaluation of the implemented benchmarks showed that the kernel bypass technique outperforms traditional userspace implementations by a wide margin. The expectation of better performance with kernel bypass was therefore fulfilled. The advantages of the kernel bypass technique described in Section 2.1.5, such as fewer context switches, reduced overhead by bypassing the network stack and direct access to the underlying hardware, could be clearly seen in the extreme performance differences in Section 5.6 compared to the userspace implementation. In addition, the benefits of using a NIC-specific driver, as described in Section 2.2.2, such as overall optimized use of the NIC and features such as hardware offloading, could be clearly seen and explained in Section 5.4 when compared to the performance of a raw socket. When comparing performance with a single flow and multiple flows, the results showed a consistent pattern of increased performance as more worker threads were added. This demonstrated the ability of the kernel bypass technique to effectively utilise multiple CPU cores when tasked with a high sending rate. The receive rate in the benchmarks scaled very well as the number of flows increased, demonstrating the efficient use of multiple cores. In addition, the effect of frame size on overall throughput was very well demonstrated. The pattern of doubling the throughput when doubling the frame size was consistent in both the kernel bypass and the userspace implementation, showing that it is better to handle fewer but larger packets as described in Section 5.3. The benchmarks themselves did not meet all expectations. In the userspace implementation, the router benchmark surprisingly performed the best of the three. In the kernel bypass implementation, the packet return and encapsulation benchmarks gave similar results, but with multiple flows the encapsulation benchmark performed better, despite being more complex than the packet return benchmark. This could be explained by the fact that the encapsulation benchmark used a different approach, as described in Section 4.2.2, due to errors in the original approach. This allowed the performance of the encapsulation benchmark to be faster because the approach of the implementation was very different, skipping the composite types entirely and manipulating the packet bits directly. A implementation of the benchmarks in the kernel space with the, in Section 2.1.1 discussed, eBPF framework would have provided another interesting performance comparison with the kernel bypass implementation as the implementation in kernel space should be faster than in user space, but still slower than the kernel bypass implementation. However, due to time constraints the implementation and evaluation of the eBPF implementation was not possible.

Building on the results of this work, further research and development can be pursued. With regard to the network function frameworks the benchmarks could be implemented in the eBPF framework, as originally planned, as well as in other frameworks such as DPDK [5] or VPP to draw a broader comparison between the performance of frameworks with similar approaches. The benchmarks themselves could also be improved, especially the routing benchmark which could deliver more realistic performance data that could be applied to real world applications. Through the creation of new benchmarks with more complex structures the packet processing capabilities could be more stressed and tested more than with the relatively simple benchmarks used in this thesis. As the thesis focused exclusively on UDP traffic, an evaluation of TCP traffic would be a great topic as it involves additional important mechanisms like flow control or congestion management which could impact the performance differently compared to UDP. Exploring the energy efficiency of NFV compared to using hardware devices in these high-speed packet processing scenarios could also contribute to more sustainable network infrastructures.

7 Conclusion

This thesis evaluated selected high-speed packet processing technologies deployed on Linux for x86 architectures. It started by presenting the concept and benefits of Network Function Virtualization, like increasing flexibility and lowering costs. The selected high-speed packet processing techniques, Snabb and eBPF, were introduced together with their respective approaches to packet processing. Additionally a traditional method in user space using Python was adapted. The advantages and disadvantages of the user space, the kernel space and the kernel bypass technique were described, and different types of Network Interface Technologies were presented. For the performance evaluation three benchmarks were implemented as network functions in the selected technologies. A packet return benchmark, a routing benchmark and a encapsulation benchmark. Each benchmark performed different operations encountered in packet processing scenarios. These built the core for evaluating the performance of the selected technologies in regards to packet processing. Datasets, containing the average performance for each benchmark in their respective implementation, were generated with different frame sizes and traffic rates and then evaluated. The results of the evaluation showed that the kernel bypass technique significantly outperforms traditional user space implementations. By utilizing the Snabb framework and its optimized driver for the mellanox NIC, substantial improvements in the performance of packet processing were demonstrated when compared to the traditional approach. This is possible as Snabb leverages hardware optimizations and reduces the overhead by bypassing the network stack. Furthermore scenarios with multiple flows were evaluated utilizing multiple CPU cores. This showed how well Snabb can scale with increasing traffic rates up to 100 Gbps by using worker threads in the implementation. The benchmarks also highlighted the importance of frame size, as large frame sizes generally led to increased performance due to the reduced overall overhead per packet. Although the implementation was successfull some parts could have been done better. The routing benchmark was not implemented optimally and the implementation of the benchmarks in eBPF could not be fulfilled. Overall this thesis achieved its goal of providing a comprehensive overview of the performance capabilities of high-speed packet processing techniques on Linux and highlighting the potential of the kernel bypass technique in achieving high performance in high-speed packet processing scenarios.

Bibliography

[1] S. Abdelwahab, B. Hamdaoui, M. Guizani, and T. Znati. Network Function Virtualization in 5G. *IEEE Communications Magazine*, 54(4):84–91, 2016.

[2] G. Ara, T. Cucinotta, L. Abeni, and C. Vitucci. Comparative Evaluation of Kernel Bypass Mechanisms for High-performance Inter-container Communications. 05 2020.

[3] T. Barbette, C. Soldani, and L. Mathy. Fast Userspace Packet Processing. In *ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*, pages 5–16, 2015.

[4] P. Biondi and the Scapy Community. Scapy. `https://scapy.net`, 2003-. Accessed: June 2024.

[5] I. Corporation. Intel dpdk: Data plane development kit. `http://dpdk.org/`. Accessed: June 2024.

[6] Data Plane Development Kit (DPDK). *Poll Mode Driver*, 2024. Accessed: June 2024.

[7] G. Davies. *Networking Fundamentals: Develop the Networking Skills Required to Pass the Microsoft MTA Networking Fundamentals Exam 98-366*. Packt Publishing, 2019.

[8] eBPF. ebpf. `https://ebpf.io`. Accessed: June 2024.

[9] N. Egi, A. Greenhalgh, M. Handley, M. Hoerdt, F. Huici, and L. Mathy. Towards High Performance Virtual Routers on Commodity Hardware. In *Proceedings of the 2008 ACM CoNEXT Conference*, page 20. ACM, 2008.

[10] E. T. S. I. (ETSI). Network Functions Virtualization (NFV); Architectural Framework. `https://www.etsi.org/deliver/etsi_gs/NFV/001_099/002/01.01.01_60/gs_NFV002v010101p.pdf`, October 2013. Accessed: June 2024.

[11] D. P. García. Snabb Explained in Less than 10 Minutes. `https://blogs.igalia.com/dpino/2017/11/13/snabb-network-toolkit/`, Nov. 2017. Accessed: June 2024.

[12] J. Gil Herrera and J. F. Botero. Resource Allocation in NFV: A Comprehensive Survey. *IEEE Transactions on Network and Service Management*, 13(3):518–532, 2016.

[13] L. Gorrie. Snabb Switch in a Nutshell. `https://github.com/lukego/blog/issues/10`, 2015. Accessed: June 2024.

[14] Gorrie, Luke. Snabb: Simple and Fast Packet Networking. `https://github.com/snabbco/snabb`. Last accessed: June 2024.

[15] B. Han, V. Gopalakrishnan, L. Ji, and S. Lee. Network Function Virtualization: Challenges and Opportunities for Innovations. *IEEE Communications Magazine*, 53(2):90–97, 2015.

[16] M. Hausenblas. *Learning Modern Linux*. O'Reilly Media, 2022.

[17] E. Kohler. The Click Modular Router. `https://github.com/kohler/click`, 2024. Accessed: 2024-06-28.

[18] M. Krasnyansky, M. Yevmenkin, and F. Thiel. *Universal TUN/TAP Device Driver*, 2002. Linux, Solaris drivers; FreeBSD TAP driver; Revision of this document by Florian Thiel.

[19] S. Kumar, A. Noor, B. K. Kaushik, and B. Kumar. Design of Ternary Content Addressable Memory (TCAM) with 180 nm. In *2011 International Conference on Devices and Communications (ICDeCom)*, pages 1–5, 2011.

[20] G. Lettieri, V. Maffione, and L. Rizzo. A Survey of Fast Packet I/O Technologies for Network Function Virtualization. pages 579–590, 10 2017.

[21] S. Lindner, M. Häberle, and M. Menth. P4TG: 1 Tb/s Traffic Generation for Ethernet/IP Networks. *IEEE Access*, PP:1–1, 01 2023.

[22] J. Madieu. *Linux Device Drivers Development*. Packt Publishing, 2017.

[23] R. Mijumbi, J. Serrat, J. Gorricho, N. Bouten, F. De Turck, and R. Boutaba. Network Function Virtualization: State-of-the-Art and Research Challenges. *IEEE Communications Surveys and Tutorials*, 18, 09 2015.

[24] G. MIRJALILY and Z. LUO. Optimal Network Function Virtualization and Service Function Chaining: A Survey. *Chinese Journal of Electronics*, 27:704–717, 07 2018.

[25] F. Paganelli, P. Cappanera, and G. Cuffaro. Tenant-defined service function chaining in a multi-site network slice. *Future Generation Computer Systems*, 121:1–18, 2021.

[26] S. Project. Performance Tuning. `https://github.com/snabbco/snabb/blob/master/src/doc/performance-tuning.md`, 2024. Accessed: June 2024.

[27] D. Scholz, P. Emmerich, and D. Raumer. Diving into Snabb. 2016.

[28] W. R. Stevens, B. Fenner, and A. M. Rudoff. *The Sockets Networking API: UNIX® Network Programming Volume 1*. Addison-Wesley Professional, 3rd edition, 2003.

[29] D. Telekom. Deutsche Telekom Tests TeraStream, the Network of the Future, in Croatia. `http://www.telekom.com/media/company/168008`. Accessed: June 2024.

[30] B. Ward. *How Linux Works, 3rd Edition: What Every Superuser Should Know*. No Starch Press, 3rd edition, 2021.

[31] T. Zhang, L. Linguaglossa, P. Giaccone, L. Iannone, and J. Roberts. Performance Benchmarking of State-of-the-Art Software Switches for NFV. *Computer Networks*, 188:107861, 2021.

List of Figures

2.1 Exemplary Snabb apps [13]. 6
2.2 Exemplary App Graph [13]. 7

4.1 Flowchart of packet return network function. 13
4.2 Flowchart of routing network function. 14
4.3 Flowchart of encapsulation network function. 15
4.4 TAP interface setup. 17

5.1 Environment for generating datasets. 21
5.2 Process of generating datasets for each benchmark. 22
5.3 Performance of packet return benchmark in Python with different network interface technologies. 23
5.4 Python benchmark performance with Raw sockets and TAP interfaces. 24
5.5 Snabb benchmark performance with one Flow at 100 Mbps sending rate. 25
5.6 Snabb benchmark performance with one Flow at 1 Gbps sending rate. 26
5.7 Snabb benchmark performances with one flow at 10 Gbps sending rate. 27
5.8 Snabb benchmark performances with one flow at 100 Gbps sending rate. 28
5.9 Snabb Snabb Mellanox driver performance compared to raw socket. . 29
5.10 Snabb Mellanox driver performance with active worker threads. . . . 30

List of Tables

4.1 Exemplary routing table. 13

5.1 Best returning rate for each benchmark in User Space and Kernel Bypass at 1024 byte frame size and 100 Gbps traffic rate. 31